Thammasat University

AI Evaluation and Model Selection Equations Explanations

Meaning, Variables and Worked Calculations
From Inputs to a Supported Model Choice

Pacharapol Pimpa

Software Engineering · Bachelor's Degree Program

Independent Technical Companion — Not Peer Reviewed

Version 1.0.0 · 13 October 2026

Purpose of this equation guide

This equation guide explains what each symbol means, what inputs are needed, how to perform the calculation and how to use the answer. It covers the six original equations and their supporting formulas, followed by a complete workflow and worked examples. Examples are explicitly instructional; they do not validate a new optimizer or claim measured cost savings. Use the original manuscript [1] for the broader research context.

Keywords: AI evaluation; model selection; calculation workflow; task quality; operational cost; non-inferiority; reproducibility.

How to read and use this guide

Use Sections 1–2 to prepare inputs. In Section 3, compute quality with Equation (1). In Section 4, calculate cost with Equation (2). Section 5 calculates the bounds required by Equation (3), applied in Section 6. Section 7 explains and uses (A1)–(A3). Sections 8–9 combine the calculations into a complete workflow and example. Sections 10–11 show interpretation and error checks. Appendices provide an equation checklist, calculator recipe and derived operational allowances.

Read each equation as an input-to-output procedure. Identify its purpose and units, supply the defined inputs, perform the arithmetic, interpret the result, then apply the stated decision rule. T1–T19 provide supporting calculations. A numerical answer alone does not establish that its sampling assumptions or evidence are valid.

Example labels distinguish constructed teaching inputs from recorded Azure values. Neither is proof that the proposed confidence-bound optimizer has been validated. Preserve these labels when copying a calculation into your own report.

1. Choose the equation route and identify its inputs

Before using the equations, assign a unique ID to each complete configuration. Record its model/version, preprocessing or prompt, supported effort, decoding, output cap, backend, timeout, retries, tools and fallback. Treat a router or cascade as one configuration and measure all of its stages. These IDs specify which recorded outputs, costs and resources belong in each calculation.

Choose one task-specific quality metric and its direction. Keep quality, money and resources in separate columns. For example, binary pass quality is a proportion, rental-count MAE has count units, cost is currency over a request horizon, and latency is time for a declared workload. Do not substitute one column for another or compare unrelated task metrics as a universal AI score.

Apply the equations in this order: observed quality → projected cost → quality and operational bounds → mandatory gates → least-cost admitted choice. Use an unchanged candidate set and workload contract. If a

required input is missing or a bound is unjustified, record insufficient evidence and stop that candidate's admission calculation.

Use the weighted route only when that is the declared selection method: point-feasibility checks → (A1) utilities → (A2) score → (A3) near-optimal set. Its output is a preference-based choice. It does not inherit the confidence-bound admission requirements of Equation (3).

2. Prepare the inputs required by the equations

2.1 Freeze the scenario before viewing selection outcomes

Write the target use case in operational terms: inputs available at prediction time, required output, correctness oracle, target population, request mix and allowed execution policy. Define the reference configuration independently of held-out results. Choose a task metric, an absolute minimum quality, an acceptable degradation margin and mandatory subgroup requirements. The reference need not be the best or even admissible under the final budget; it is a declared comparison point.

Fix the horizon N, currency, tariffs and resource ceilings. State whether costs are incremental or fully allocated. Choose statistical error allocations, sampling units and analysis methods before examining candidate outputs. Margins must come from task needs or an explicitly limited research scenario. They cannot be loosened after seeing that a favored model failed.

Separate development, selection and held-out confirmation. Development tunes prompts, features and settings. Selection measures the frozen candidates and chooses a policy. Held-out confirmation checks that frozen decision on untouched cases. A held-out failure is recorded as a failure or inconclusive result; it does not authorize picking a different candidate on the same held-out data.

2.2 Keep one auditable record for each case and execution

For each case retain a unique case ID, task and split ID, input or input hash, reference outcome, candidate ID, model/version/policy hash, run ID and seed, recorded output, failure state and metric-oracle result. Retain latency, billable usage categories, retries, tool calls and resource telemetry separately. Case IDs preserve pairing: candidate and reference must be scored on the same underlying cases.

A missing output is not silently removed. Use the predefined handling rule; for a code-pass task, compilation errors, truncated output, rejected execution and timeouts can count as failures. Missing execution records that prevent comparable measurement are an insufficient-evidence state. Keep failed-request cost in the accounting even if its quality result is a failure.

Distinct rows are not automatically independent observations. Exact duplicates, related users, shared repositories, repeated generations and adjacent time-series observations may form dependent groups. Use the correct sampling unit and keep related groups within one data split. Three generations on one problem do not create three independent problems. A fixed benchmark also needs a defensible target-population interpretation; calculating a confidence interval does not establish representativeness.

3. Equation (1): calculate task-specific quality

$$Q_t^D(c) = M_t((y_i, f_c(z_i))_{i \in D}) \quad (1)$$

3.1 Meaning of every symbol and the calculation order

The task identifier t, configuration identifier c and case index i are dimensionless labels. D is the collection of case identities being evaluated; n is its size. The input z_i and reference outcome y_i have task-specific meanings and units. The frozen execution policy f_c produces the recorded output $f_c(z_i)$. The functional M_t takes the indexed sequence of all reference/output pairs and returns the declared quality metric. Equal-valued pairs at different case IDs remain in the sequence: this is not a set operation that removes duplicates.

Q_t^D(c) is an observed estimate on D. Q_t(c), without the superscript D, is the unknown population quality under the specified workload and output-randomness distribution. The superscript D identifies the evaluation data; it is not exponentiation. A measured score is not automatically equal to future-population quality.

How to use Equation (1). Inputs: aligned case IDs, reference outcomes, recorded outputs and a frozen metric/oracle. Calculation: validate coverage, score every eligible record, then apply the chosen task metric. Result: one observed quality value in the metric's declared units, with sample size and dependence recorded. Use: calculate its appropriate lower bound and its paired degradation bound before testing the quality gates in Equation (3). For classification, start with T1–T3; for regression, use T4–T5 and negate the error for a higher-is-better quality definition. Use T6–T7 separately when operating loss is required.

3.2 Classification and binary code correctness

$$Accuracy(c) = \frac{1}{n}\sum_{i=1}^{n} I(\hat{y}_i = y_i) \quad \text{(T1)}$$

Here $\hat{y}_i$ is the predicted label, I(statement) is one if the statement is true and zero otherwise, and n is the number of scored cases. Accuracy is dimensionless. For a single-completion code task, replace $I(\hat{y}_i = y_i)$ with a frozen pass/fail indicator s_i, where one means the output passes the complete declared test procedure. Then quality is x/n, with $x = \sum s_i$ successes. Tests are an imperfect correctness oracle; the meaning of a pass depends on the test suite, parser, timeout and allowed execution.

$$Precision_k = \frac{TP_k}{TP_k+FP_k}, \quad Recall_k = \frac{TP_k}{TP_k+FN_k} \quad \text{(T2)}$$

$$F1_k = \frac{2TP_k}{2TP_k+FP_k+FN_k}, \quad MacroF1 = \frac{1}{K}\sum_{k=1}^{K} F1_k \quad \text{(T3)}$$

For class k, TP_k counts correctly predicted members of k; FP_k counts other classes predicted as k; FN_k counts true members of k predicted as another class. K is the fixed number of evaluated classes. Precision, recall and F1 are proportions. Compute the full confusion matrix first, then each class's F1, then the unweighted average across the K classes [2]. Record the class list and zero-denominator convention; changing them changes what the metric measures. Macro-F1 is not an average of per-case F1 values.

Work example — two-class classification.

Given. (TP, FP, FN) = (40, 5, 10) for class A and (30, 10, 5) for class B; n = 85.

$$F_1(A) = (2 \times 40)/(2 \times 40 + 5 + 10) = 80/95 \approx 0.842105$$
$$F_1(B) = (2 \times 30)/(2 \times 30 + 10 + 5) = 60/75 = 0.800000$$
$$MacroF1 = [(80/95) + (60/75)]/2 \approx 0.821053$$
$$Accuracy = (40 + 30)/85 = 70/85 \approx 0.823529$$

Result. Macro-F1 averages class scores; accuracy counts correct cases. They measure different properties. This is illustrative arithmetic.

3.3 Regression and the higher-is-better conversion

$$MAE(c) = \frac{1}{n} \sum_{i=1}^{n} |\hat{y}_i - y_i| \quad \text{(T4)}$$

$$RMSE(c) = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2} \quad \text{(T5)}$$

Given. Targets y = (10, 20, 30); predictions ŷ = (12, 17, 31).

$$|y - \hat{y}| = (2, 3, 1)$$
$$MAE = (2 + 3 + 1)/3 = 2$$
$$RMSE = \sqrt{[(2^2 + 3^2 + 1^2)/3]} = \sqrt{(14/3)} \approx 2.160247$$

Result. Both results use the outcome's units. RMSE gives greater weight to larger errors [3,4].

Given. Maximum acceptable MAE = 60 counts; reference MAE = 50; candidate MAE = 52.

$$Q = -MAE,\ q_{min} = -60$$
$$Q_{ref} = -50,\ Q_c = -52$$
$$\Delta = Q_{ref} - Q_c = -50 - (-52) = 2 \text{ counts}$$

Result. The positive degradation means the candidate is worse. Negating error preserves its units and makes higher quality better; it does not turn error into a proportion.

3.4 Quality metrics and operating loss

$$\ell_i = \omega_{y_i} I(\hat{y}_i \neq y_i) \quad \text{(T6)}$$

$$\ell_i = \lambda_{under} max(y_i - a_i, 0) + \lambda_{over} max(a_i - y_i, 0) \quad \text{(T7)}$$

In T6, the true-class weight ω (class y_i) is the declared penalty for misclassifying the true class y_i; it is dimensionless in the historical simulation. In T7, a_i is the decision or nonnegative point prediction, λ_{under} and λ_{over} are nonnegative penalty weights, and ℓ_i is simulated weighted-count loss. The average is $\sum \ell_i$/n. These are the historical operating-loss definitions; they are not monetary costs or universal quality scores.

Given. True count y = 20; underprediction weight = 2; overprediction weight = 1.

$$\ell(20, 17) = 2\ max(20 - 17, 0) + max(17 - 20, 0) = 6$$
$$\ell(20, 23) = 2\ max(20 - 23, 0) + max(23 - 20, 0) = 3$$

Result. Equal absolute errors can have different operating losses. If the task requires both MAE and operating loss, state both requirements.

4. Equation (2): calculate complete operational cost

$$C_N(c) = C_{setup}(c) + N \sum_h p_h E[v_h(c)] + C_{support,N}(c) \quad (2)$$

4.1 Units and an accounting ledger

N counts requests over one deployment horizon. h indexes non-overlapping billable categories. p_h is currency per billable unit; v_h(c) is billable units per request for the complete policy. E denotes expectation over the declared request mix and execution randomness. Each product p_h E[v_h] is currency/request. Multiplying by N produces currency, so it can be added to C_{setup}(c) and $C_{support}, N$(c), both allocated to the same horizon.

C_{setup} covers an allocated loading, integration or training expense. $C_{support}, N$ covers infrastructure charges not already included elsewhere. Record the accounting boundary: full allocation and incremental spending are different estimates. If the same paid node time contains loading and serving, allocate it once. Do not add loading as setup while also charging the identical minutes in serving rental.

Create categories for cached and uncached input tokens, billed output, compute rental, paid tool invocations, transfer and other applicable charges. Include verification, routing, failed calls, retries and fallbacks in the complete policy's usage. Reasoning tokens already included in billed output must not be charged a second time. For self-hosted inference, tokens are resource telemetry while node/GPU rental is the monetary charge; do not add a hypothetical API token fee.

4.2 Estimate usage from measured requests

$$\bar{v}_h(c) = \frac{1}{m} \sum_{j=1}^{m} v_{h,j}(c) \quad \text{(T8)}$$

Here m is the number of cost-profiling requests, j identifies one measured request, and v_h, j is its recorded category-h consumption across all attempts. The sample mean estimates E[v_h] only for the sampled workload. m and N have different jobs: m describes measurement evidence; N describes the future accounting horizon. If category usage is paired with candidate/reference requests, preserve that pairing in comparative analyses.

For a stratified workload, let g identify an input-defined group, π_g its future request proportion and μ_h, g the estimated mean category consumption within g. Replace the pooled mean by $\sum_g \pi_g \mu_h, g$, with nonnegative π_g summing to one. Define groups before seeing model answers and sample enough requests per group. Uncertainty in π_g and the group means must be reflected in planning bounds; a changed workload mix invalidates a previous pooled estimate.

If a tariff quotes USD 0.50 per million tokens, convert it to USD 0.00000050 per token before multiplication. Token counts, milliseconds and node-hours cannot share a category without conversion. Use actual billable counts and stated rounding/minimum-charge rules. Piecewise tariffs, variable exchange rates or nonlinear discounts need a ledger that reflects those rules instead of forcing an inappropriate constant-price approximation.

4.3 A transparent illustrative calculation

Given. Teaching assumptions: N = 1,000 requests; setup = USD 1; support = USD 2; mean input = 1,000 tokens/request; mean output = 250 tokens/request.

$$p_{in} = 0.50/10^6 = 0.0000005 \text{ USD/token}$$
$$p_{out} = 2/10^6 = 0.000002 \text{ USD/token}$$

$$C_{request} = 1{,}000(0.0000005) + 250(0.000002) = 0.001 \text{ USD}$$
$$C_N = 1 + 1{,}000(0.001) + 2 = 4 \text{ USD}$$

Result. Equation (2) adds setup, the workload's expected variable charge and support. These prices are teaching assumptions, not provider quotations or invoices; the ledger must include every paid attempt.

How to use Equation (2). Inputs: request horizon, one currency, unit-normalized tariffs, representative full-policy usage, allocated setup and supporting charges. Calculation: multiply each price by its expected usage, sum category costs per request, multiply by the horizon, then add setup and support. Result: projected total cost; the preceding example gives USD 4. Use: obtain a separately justified upper allowance for the budget gate, then rank only candidates passing every gate by projected cost. Include failed requests and every retry, tool or fallback. If usage already covers all attempts, do not multiply it again by a retry factor. Reconcile the projection with actual billing separately.

4.4 Projection, upper allowance and invoice

C_N is an expected-cost projection. C_N^U is a separately justified upper allowance; its superscript U means upper, not a power. It may bound expected cost, realized future-horizon expenditure or an enforced deterministic cap. State which object it bounds. A flat 20% contingency is a planning convention and has no automatic probability coverage.

Actual billed expenditure comes from provider invoices or usage reconciliation, including delayed supporting charges, discounts and credit treatment. Student credit can affect out-of-pocket expenditure without changing the resource consumption or advertised tariff. Compare candidates on the same accounting basis. The historical Azure warm CPU-dollar proxy is a partial calculation and cannot substitute for complete operational cost.

5. Supporting equations: calculate the required bounds

5.1 Absolute binary-quality bounds

For fixed-size independent, identically distributed problem-level binary outcomes, use x successes in n problems. Let $B_{inv}(q;a,b)$ be the q-quantile of a Beta distribution with positive shape parameters a and b. A claim's one-sided error allocation is e, not automatically the global family allowance α. Clopper–Pearson bounds [5] are calculated as follows:

$$L_e(x,n) = B_{inv}(e; x, n - x + 1), \quad 0 < x \leq n \quad \text{(T9)}$$

$$U_e(x,n) = B_{inv}(1 - e; x + 1, n - x), \quad 0 \leq x < n \quad \text{(T10)}$$

Set $L_e(0,n) = 0$ and $U_e(n,n) = 1$ rather than evaluating undefined Beta shapes. Require integer $n>0$, integer $0 \leq x \leq n$ and $0<e<1$. These limits are frequentist coverage procedures, not statements that the unknown quality is a random parameter with a posterior probability. Larger samples generally make the bounds more informative, but representativeness and independence remain essential.

For a practical implementation, SciPy's beta.ppf computes the required quantile [6]. Specifically, use beta.ppf(e,x,n−x+1) for the lower bound and beta.ppf(1−e,x+1,n−x) for the upper bound, with the boundary cases above. Preserve unrounded values for admission decisions; round only when displaying results.

5.2 Reference-relative quality must be paired

$$\Delta = Q_t(c_{ref}) - Q_t(c) = p_{10} - p_{01}, \quad \hat{\Delta} = \frac{n_{10} - n_{01}}{n} \quad \text{(T11)}$$

For binary pass quality, n_{10} counts reference-pass/candidate-fail cases; n_{01} counts reference-fail/candidate-pass cases. n_{11} counts both pass; n_{00} counts both fail. All four counts sum to n. p_{10} and p_{01} are the corresponding unknown population probabilities. The same case IDs must be used for both configurations. A positive Δ is candidate degradation, and a negative Δ is improvement on this metric.

The paired count identity follows because reference success count is $n_{11}+n_{10}$ and candidate success count is $n_{11}+n_{01}$. Their difference is $n_{10}-n_{01}$. Equal total successes can conceal many disagreements, so retain the paired table rather than only the two overall proportions.

$$U_{\Delta,e} = U_{e/2}(n_{10}, n) - L_{e/2}(n_{01}, n) \quad \text{(T12)}$$

The two discordance counts are dependent, but each marginal count is binomial under independent, identically distributed problem-level sampling. With failures totaling at most e by the union bound, p_{10} is below its upper limit and p_{01} above its lower limit; subtracting yields a conservative upper degradation bound. This construction uses pairing without assuming the two categories or candidate policies are independent. It can be conservative and low-powered. Admission requires an upper degradation bound at or below δ; a nonsignificant difference from zero is insufficient.

Given. Two distinct policies agree on all observed cases; e = 0.05/7.

$$U_{\Delta} = 1 - (e/2)^{1/n}$$
$$n = 90 \Rightarrow U_{\Delta} \approx 0.060689$$
$$n = 390 \Rightarrow U_{\Delta} \approx 0.014344$$

Result. Zero sample disagreements do not imply zero population degradation. Only an exact same-policy comparison on the same records has degradation zero by identity.

5.3 Allocate error before candidate selection

If four configurations need absolute-floor claims and three non-reference configurations need degradation claims, there are seven quality claims. Equal Bonferroni allocation uses e = α/7. With α=0.05, e=0.007142857, and each paired marginal receives e/2=0.003571429. This controls the family under the stated coverage assumptions without requiring independence across candidates.

If the registry changes, additional metrics or subgroups become mandatory, or data are inspected repeatedly with adaptive stopping, the allocation/design must change. The exact count seven is an example, not a universal constant. Held-out confirmation of one frozen winner can use two quality claims, with e=0.05/2 when a separate 0.05 family is declared. That family does not retroactively certify the selection sample.

For macro-F1, recompute the full confusion-matrix metric in each defensible resample. For paired regression, resample shared case/groups or time blocks and recompute both metrics and their difference. A bootstrap distribution of an inappropriate sampling unit is not repaired by increasing the draw count. These approximate intervals need justified coverage and dependence assumptions; the binomial formulas above do not apply directly to macro-F1 or rental-count MAE.

5.4 Calculate an upper cost allowance

The following optional bounds illustrate a fully stated calculation method rather than adding another central framework equation. Suppose m independent, identically distributed profiling requests have category usage in a known fixed range $0 \le v_h, j \le V_h$. A one-sided bounded-variable allowance based on Hoeffding's inequality [7] is:

$$\mu_h^U = min(V_h, \bar{v}_h + V_h \sqrt{\frac{ln(1/\eta_h)}{2m}}) \quad \text{(T13)}$$

$$C_N^{U,mean} = C_{setup}^U + N \sum_h p_h \mu_h^U + C_{support,N}^U \quad \text{(T14)}$$

Here V_h is a justified per-request usage cap in category units and η_h is the allocated error for that mean. Prices are fixed and nonnegative. Setup/support upper bounds must also be justified. Allocate error across categories and uncertain fixed charges. T14 bounds expected horizon cost, not a realized invoice. A cap inferred from the observed maximum alone does not satisfy T13's assumptions.

If the future N requests have independent identically distributed nonnegative variable costs S_i bounded by a known S_{max}, with future sampling matching profiling and a valid upper expected variable cost μ_S^U, a separate predictive allowance is:

$$C_N^{U,real} = C_{setup}^U + N \mu_S^U + S_{max} \sqrt{\frac{N \, ln(1/\beta)}{2}} + C_{support,N}^U \quad \text{(T15)}$$

β allocates error to future-horizon fluctuation. Profiling-mean and future-fluctuation failures must be combined with any setup/support errors. Correlated request bursts, drifting tariffs and unbounded retries violate these simple assumptions. A validated deterministic billing cap is another option, but the stop/fallback policy and its quality consequences must be evaluated. If no defensible method applies, report insufficient cost evidence.

5.5 Latency percentiles and other resource limits

Measure end-to-end request latency with a monotonic clock, including the frozen policy's retries and verification. State concurrency, load, hardware, region, warm/cold treatment, measurement unit and timeout handling. Do not mix a 32-case batch latency with a per-request interactive limit. A descriptive empirical p95 is not automatically an upper confidence bound on population p95.

For an explicit optional percentile allowance, suppose m profiling latencies are independent draws from one continuous, stationary distribution. Sort them T_(1)≤...≤T_(m). Let p=0.95 and γ be the error for the percentile claim. Choose the smallest integer k from 1 to m satisfying:

$$Pr(Binomial(m,p) \ge k) \le \gamma, \quad r_{latency}^U = T_{(k)} \quad \text{(T16)}$$

Binomial(m,p) counts observations below the true p-quantile. Except when that count is at least k, the k-th order statistic is at or above the population quantile. This explains the one-sided upper allowance. For m=100 and γ=0.05, k=99. For m=20 there is no qualifying k, so this finite-sample construction returns insufficient evidence. For m=59, the sample maximum k=59 is just sufficient. Censoring/timeouts require a predefined compatible analysis; treating a timeout as a short successful response invalidates the bound.

Memory maxima sampled intermittently can miss spikes. An enforced allocation limit is different from a statistical allowance on sampled peak RSS. A memory cap that causes crashes must include those failures in quality measurement. For token quotas or rate limits, evaluate full-policy usage under the declared

concurrency and horizon. Every resource statistic r_l^U and ceiling b_l must match in definition and unit. Quality-family α does not automatically cover resource or cost claims; a single combined probability needs allocations across all stochastic gates.

6. Equation (3): from evidence to model selection

$$c^* = arg\,min_{c \in \mathcal{C}_t} C_N(c)$$

$$s.t. \quad L_\alpha[Q_t(c)] \geq q_{min}, \quad U_\alpha[Q_t(c_{ref}) - Q_t(c)] \leq \delta$$

$$C_N^U(c) \leq B_N, \quad r_l^U(c) \leq b_l, \quad \forall\, l \quad (3)$$

6.1 Objective and all four gates

The finite set $\mathscr{C}$_t contains supported, compatible candidates with complete and comparable evidence for the frozen scenario. The broader registry also retains incompatible or incomplete candidates with reasons. c* denotes the selected configuration; argmin mathematically returns the set of minimum-cost configurations, and the tie rule below chooses one member. C_N is projected expected cost over the specified N-request horizon.

Gate 1, L_α[Q_t(c)]≥q_{min}, requires evidence that absolute quality meets the floor. The displayed α names the declared family policy; implementation must use the appropriate allocated claim error e. Gate 2, U_α[$Q_t(c_{ref}$)−Q_t(c)]≤δ, limits reference-relative degradation in the same metric units. For a proportion, δ=0.05 means five percentage points, not a five-percent relative reduction. For negative MAE, δ is in original outcome units.

Gate 3, $C_N^U \leq B_N$, compares the declared cost allowance with the budget in the same currency and horizon. Its target—expected cost, future expenditure or deterministic cap—must be stated. Gate 4, $r_l^U \leq b_l$ for every l, checks each mandatory resource definition and ceiling. Here l is a resource label; b_l is positive; r_l^U is the corresponding justified allowance. A superscript U identifies an upper quantity, never exponentiation.

How to use Equation (3). Inputs: compatible complete candidates, an absolute quality floor, reference-relative margin, justified quality bounds, budget allowance and resource bounds. Calculation: check each gate separately, combine all gate outcomes with logical AND, then minimize projected cost within the admitted set. Result: one frozen configuration after deterministic ties, or no supported selection. Use: issue a decision receipt and confirm that frozen configuration on untouched cases. Passing the absolute floor does not excuse excessive reference-relative degradation; closeness to a weak reference does not excuse failing the floor. The reference itself must still meet absolute quality and operations.

6.2 Read the decision status

Admissible means every mandatory gate has been supported. Operationally rejected means a supported cost/resource allowance violates a limit. Insufficient evidence means a required measurement, bound assumption or comparable record is missing or invalid. Inconclusive quality evidence means the available bound does not establish admission; it does not automatically prove true population quality is unacceptable. Proven quality violation requires an appropriate directional analysis for that different claim. Incompatible means the configuration cannot support the task or requested policy.

Compute these outcomes before ranking. Do not substitute zero for unknown cost, grant a pass to a missing bound, or delete unfavorable failures. Preserve all rejection reasons. If an operational limit is a deterministic prohibition, no quality advantage can compensate for it.

6.3 Deterministic tie-breaking and optional resource use

$$\rho(c) \;=\; max_{l}\, \frac{r_l^U(c)}{b_l} \quad \text{(T17)}$$

For an admissible candidate, each ratio is dimensionless; smaller ρ indicates more headroom for the declared resources. It is not a task-performance score. The master objective remains monetary cost. Break exact projected-cost ties by smaller conservative cost, then optionally smaller ρ, then the immutable configuration ID in a documented order. Use a fixed total ordering.

If prices are rounded to cents or a cost tolerance is used, freeze that convention first and call the output near-minimum when appropriate. Pairwise epsilon comparisons can be nontransitive; use a defined set relative to one global minimum, not an arbitrary sequence of pairwise ties. Minimizing C_N^U as the primary objective would be a separately named conservative-cost variant, not the displayed Equation (3).

7. Equations (A1)–(A3): explanation and step-by-step use

7.1 Equation (A1): turn each criterion into a utility

$$u_j(m,t,s) \;=\; clip_{[0,1]}\left(\frac{x_j - a_j}{b_j - a_j}\right) \quad \text{(A1)}$$

The predictive pipeline is m, the task t, the simulated operating profile s and the criterion j. This m is an identifier, distinct from the profiling sample-size m used in Sections 4–5; the context must be retained. The measured criterion x_j has native units. a_j is its unfavorable anchor and b_j its favorable anchor, in matching units, with $a_j \neq b_j$. The utility u_j(m,t,s) is dimensionless. Clipping v to [0,1] means max(0,min(1,v)).

Given. Fixed bad/good loss anchors = 0.30/0.02; loss = 0.10. Fixed bad/good latency anchors = 50/0.5 ms; latency = 2.5 ms.

$$u_{loss} = (0.10 - 0.30)/(0.02 - 0.30) = 5/7 \approx 0.714286$$
$$u_{latency} = (2.5 - 50)/(0.5 - 50) = 95/99 \approx 0.959596$$
$$u_{clipped} = max[0,\, min(1,\, u)]$$

Result. For a lower-is-better criterion, the good anchor is smaller. Clipping loses distinctions beyond the anchors; these utilities express preferences, not statistical calibration.

How to use (A1). Inputs: a measured criterion and distinct unfavorable/favorable anchors in matching units. Calculation: subtract the unfavorable anchor, divide by the favorable-minus-unfavorable span, then clip between zero and one. Result: a dimensionless utility. Use: pass that utility to (A2), preserving the criterion's declared weight. Freeze anchors before comparison. The preceding numbers illustrate historical anchor definitions; they are not newly measured model outcomes. Clipping changes the preference representation and does not reduce measurement uncertainty.

7.2 Equation (A2): compute the weighted preference score

$$S(m,t,s) \;=\; 100 \sum_{j} w_j(t,s)\, u_j(m,t,s),\; for\; F(m,t,s) \;=\; 1 \quad \text{(A2)}$$

w_j(t,s) is a nonnegative criterion weight and the weights sum to one. F(m,t,s)=1 means the historical pipeline passes all original point-evidence hard gates with complete records. S(m,t,s) is measured in preference score points from 0 to 100. F is not the new confidence-based admissibility rule. A rejected candidate has no compensatory eligible score, rather than a conveniently ranked zero.

Given. Use the full-precision utilities 5/7 and 95/99; weights are fixed, nonnegative and sum to one.

$$S_{quality} = 100[0.8(5/7) + 0.2(95/99)] \approx 76.334776 \text{ points}$$
$$S_{resource} = 100[0.3(5/7) + 0.7(95/99)] \approx 88.600289 \text{ points}$$

Result. For (A2), check feasibility first, multiply each utility by its weight, sum and multiply by 100. Different preferences produce different scores. These points are neither currency nor universal intelligence; retain full precision until display.

7.3 Equation (A3): form the near-optimal set and pick deterministically

$$A_{\varepsilon} \;=\; \{m :\; F(m,t,s) \;=\; 1\; and\; S(m,t,s) \geq S_{max} \;-\; \varepsilon\} \quad \text{(A3)}$$

S_{max} is the highest score among feasible pipelines and ε≥0 is a score-point tolerance. The near-optimal set is the set of feasible pipelines whose score lies no more than ε below S_{max}. It is not the set within ε dollars of the cheapest model. The historical protocol uses ε=0.5 points and the fixed order linear, forest, boosting to choose among members.

Given. Feasible scores: linear = 76.4, forest = 76.1, boosting = 76.8; ε = 0.5; fixed order = linear, forest, boosting.

$$S_{max} = max(76.4, 76.1, 76.8) = 76.8$$
$$S_{cutoff} = 76.8 - 0.5 = 76.3$$
$$\mathcal{A}_{\varepsilon} = \{linear, boosting\} \Rightarrow c^{*} = linear$$

Result. For (A3), find one global maximum, subtract ε, retain every feasible score at or above the cutoff, then apply the fixed order. Record the set and choice. An empty feasible set returns no selection; its maximum is undefined.

8. Apply the equations in one complete workflow

8.1 Define, freeze and measure

1. Define the calculation contract.

$$Contract = (t, M_t, q_{min}, \delta, c_{ref}, N, B_N, \{b_l\})$$

Use. Specify the population, inputs, outputs, complete policies, quality oracle, currency and units. Freeze evidence requirements, dependence assumptions, error allocation, failure handling and tie order.

2. Separate the datasets and freeze the policies.

$$D_{dev} \cap D_{sel} = D_{dev} \cap D_{test} = D_{sel} \cap D_{test} = \varnothing$$

Use. Keep duplicate or related groups together. Tune only on development data. Before selection, freeze versions, prompts/features, tools, supported effort, decoding, caps, sandbox, seeds, tariffs and metrics. For human ratings, freeze a rubric, blinding and agreement checks.

3. Run each complete policy on the same selection cases.

$$\hat{y}_i(c) = f_c(z_i),\ i \in D_{sel}$$
$$Record_i(c) = (ID_i, y_i, \hat{y}_i, failure_i, usage_i, latency_i, resources_i)$$

Use. Retain failures, every paid attempt, token categories where supplied, tool calls, retries and fallback transitions. Use separate profiling sessions when measurement interference would distort operations.

4. Check evidence completeness and comparability.

$$IDs(c) = IDs(c_{ref}) = D_{sel}$$
$$Coverage = recorded\ required\ cases\ /\ all\ required\ cases = 1$$

Use. Coverage alone is insufficient. Also verify references, allowed inputs, units, counts, hashes and absence of held-out leakage. Mark missing or incompatible evidence explicitly; a confidence formula cannot repair data errors.

5. Calculate the task metric and, if needed, operating loss.

$$Q_t^D(c) = M_t((y_i, f_c(z_i))_{i \in D})$$
$$Mean\ operating\ loss = (1/n)\ \Sigma_i\ \ell(y_i, \hat{y}_i)$$

Use. Use x/n and paired counts for binary pass quality, the confusion matrix for macro-F1, or MAE/RMSE for regression. Keep outcome units and record operating loss separately.

6. Apply the frozen uncertainty method and error allocation.

$$Quality\ evidence = (L_e[Q_t(c)],\ U_e[Q_t(c_{ref}) - Q_t(c)])$$

Use. For the binary route use T9–T12, including endpoint rules. For dependent records use a defensible group/time method or narrow the claim. Confirm the degradation sign and units before comparing with δ.

7. Calculate expected cost and a justified budget allowance.

$$C_N(c) = C_{setup}(c) + N\ \Sigma_h\ p_h\ E[v_h(c)] + C_{support,N}(c)$$
$$Budget\ input = C_N^U(c)$$

Use. Use a non-overlapping, workload-representative ledger with setup, support, failures, tools, routing and fallback. State whether the allowance targets expected cost, realized expenditure or an enforced cap.

8. Match each measured resource to its ceiling.

$$Resource\ evidence = \{r_l^U(c)\},\ resource\ limits = \{b_l\}$$

Use. Use matching units, concurrency and statistics. Describe warm/cold behavior, justify allowances or enforced caps, retain failure consequences and allocate error separately for stochastic operational claims.

8.2 Admit, select and confirm

9. Admit a candidate only when every supported gate passes.

$$Admit(c) = [L_\alpha(Q) \geq q_{min}] \wedge [U_\alpha(\Delta) \leq \delta]$$
$$\wedge [C_N^U(c) \leq B_N] \wedge [r_l^U(c) \leq b_l \text{ for every } l]$$

Use. Admission also requires complete, comparable evidence. Preserve missing, inconclusive and violating statuses with reasons. Only admitted candidates enter the cost ranking.

10. Rank admitted configurations by the declared objective.

$$\mathcal{A} = \{c \in \mathcal{C}_t : Admit(c)\}$$
$$c^* \in \arg\min_{c \in \mathcal{A}} C_N(c)$$
$$\mathcal{A} = \varnothing \Rightarrow \text{no supported selection}$$

Use. Apply the fixed tie rule and record the winner and runner-up. If the set is empty, identify the evidence or scenario change needed. Do not relax thresholds after seeing results.

11. Confirm the frozen decision on independent held-out cases.

$$D = D_{test} \Rightarrow \text{recompute the prespecified quality and operational evidence}$$

Use. Use the declared held-out error family and baselines. Preserve adverse or inconclusive findings. Confirmation is not reselection; any fallback must be prespecified and satisfy its own requirements.

12. Reconcile billing and retain the decision record.

$$\text{Cost deviation} = C_{billed,N} - C_N(c^*)$$
$$\text{Receipt} = (\text{inputs, hashes, calculations, bounds, gate reasons, winner, confirmation})$$

Use. $C_{billed,N}$ is actual billed expenditure on the same horizon and accounting basis. Retain immutable artifacts and monitor drift. Changed model aliases, routing, tariffs or request mix require a new evaluation.

The historical alternative replaces Steps 6, 9 and 10 with its original point-feasibility gates, A1 utilities, A2 score and A3 near-optimal set. It does not thereby acquire the statistical guarantees or full-cost semantics of the new proposal.

9. Worked example: inputs, calculations and final selection

9.1 Teaching inputs, not new model evidence

Everything in this section is deliberately constructed arithmetic. No named AI model was tested and no price is a current quotation. The example demonstrates how a calculator applies the proposal; it cannot validate cost savings or quality preservation in a real service. Statistical calculations assume 1,000 independent identically distributed binary task outcomes. Cost/resource upper values are stipulated valid allowances for the exercise; a real evaluation would need the profiling evidence and bound method supporting them.

The registry has four complete hypothetical policies: reference R and candidates A, B and C. Another policy D lacks required records and is retained as insufficient evidence outside the evidence-ready candidate set. Freeze q_{min}=0.80, δ=0.05, quality-family α=0.05, N=1,000 future requests, B_N=USD 8, latency ceiling 500 ms at request p95 and memory ceiling 1,024 MiB. The 1,000 scored problems and 1,000 future requests happen to have equal counts; their roles are different. Seven quality claims give e=0.05/7.

9.2 Calculate point quality and check paired consistency

Given. Teaching case: n = 1,000; paired tuple order is (n_{11}, n_{10}, n_{01}, n_{00}); R passes 850 cases.

$$A: (830, 20, 10, 140) \Rightarrow x_A = 830 + 10 = 840$$
$$B: (845, 5, 35, 115) \Rightarrow x_B = 845 + 35 = 880$$
$$C: (760, 90, 20, 130) \Rightarrow x_C = 760 + 20 = 780$$
$$\Sigma\, n_{ab} = 1{,}000;\ n_{11} + n_{10} = 850 \text{ in every row}$$

Result. Each row is internally consistent and uses the same reference outcomes. These are constructed counts, not experimental observations.

Given. Use the paired counts and divide by n = 1,000.

$$(Q_R^D, Q_A^D, Q_B^D, Q_C^D) = (0.850, 0.840, 0.880, 0.780)$$
$$\Delta_A^D = (20 - 10)/1{,}000 = 0.010$$
$$\Delta_B^D = (5 - 35)/1{,}000 = -0.030$$
$$\Delta_C^D = (90 - 20)/1{,}000 = 0.070$$

Result. B has the highest observed pass quality. Point estimates describe the sample; admission requires uncertainty bounds.

9.3 Calculate the absolute and degradation bounds

Given. Apply T9 with e = 0.05/7 and $q_{min} = 0.80$.

$$(L_R, L_A, L_B, L_C) \approx (0.820311, 0.809621, 0.852651, 0.746214)$$
$$L_R, L_A, L_B \geq 0.80;\ L_C < 0.80$$

Result. R, A and B establish the floor by this rule. C is not admitted; failing to establish the floor is not by itself proof of population failure.

Given. Apply paired preservation bounds with $\delta = 0.05$; calculate at full precision before rounding.

$$U_\Delta(A) = 0.035127834\ldots - 0.003547662\ldots \approx 0.031580171 \leq 0.05$$
$$U_\Delta(B) = 0.014572887\ldots - 0.021291258\ldots \approx -0.006718372 \leq 0.05$$
$$U_\Delta(C) = 0.117011669\ldots - 0.010076291\ldots \approx 0.106935378 > 0.05$$
$$U_\Delta(R) = 0 \text{ by same-policy identity}$$

Result. A and B support the preservation requirement; C does not. The same-policy identity for R is not evidence of equivalence between different policies.

Use unrounded values for the decisions; the displayed six or nine decimals are presentation only. If the records were clustered or adaptively collected, these numerical Beta quantiles would still be computable but their advertised binomial coverage would not follow.

9.4 Calculate costs and operational gates

Given. Teaching ledgers: N = 1,000; all monetary values are USD.

$$C_N(A) = 1 + 1{,}000(0.001) + 2 = 4$$
$$C_N(B) = 2 + 1{,}000(0.003) + 2 = 7$$
$$C_N(C) = 0.50 + 1{,}000(0.0005) + 1 = 2$$
$$C_N(R) = 2 + 1{,}000(0.005) + 2 = 9$$

Result. Each line adds setup, expected variable cost and support. These hypothetical full-ledger totals are not invoices.

Given. Order (A, B, C, R); ceilings are USD 8, latency 500 ms and memory 1,024 MiB. Appendix C derives the exercise allowances.

$$C_N^U = (5, 8, 3, 10)\ \text{USD}$$
$$r_{latency}^U = (350, 450, 200, 600)\ \text{ms}$$
$$r_{memory}^U = (512, 768, 256, 512)\ \text{MiB}$$
A, B, C: all operational gates pass; R: cost and latency gates fail

Result. A reference may remain a useful quality benchmark even when it is operationally inadmissible.

D has no comparable bounds and must not be assigned a free cost or perfect quality. It is not ranked. In a real decision, unsupported cost/resource allowances for A or B would likewise produce insufficient evidence, regardless of the arithmetic in this exercise.

9.5 Select and interpret

Given. After all supported gates, the admissible set is $\mathcal{A} = \{A, B\}$.

$$C_N(A) = 4\ \text{USD} < C_N(B) = 7\ \text{USD} \Rightarrow c^* = A$$
$$\rho(A) = \max(350/500, 512/1{,}024) = 0.70$$
$$\rho(B) = \max(450/500, 768/1{,}024) = 0.90$$

Result. Resource ratios are optional tie-break information; costs already differ. C is cheaper but not admitted. B's higher observed quality does not override the least-cost objective.

Given. Reference projected cost = USD 9; selected A projected cost = USD 4.

$$\text{Projected difference} = 9 - 4 = 5\ \text{USD}$$
$$\text{Relative difference} = (5/9) \times 100\% \approx 55.5556\%$$

Result. This is only a constructed calculation, not a demonstrated research improvement. Actual billed savings and held-out confirmation are absent. A real deployment would first freeze A and run the prespecified independent confirmation.

10. Use the equations to interpret reported values

Use reported values [1] to practise interpretation, while keeping the selected quality metric separate from operating loss and resource cost. The following calculations show metric differences, paired-loss signs, interval precision and bootstrap endpoints. Reconstructing an uncertainty interval still requires its original case-level records and resampling assumptions.

Given. Historical reported values in order (linear, forest, boosting).

$$\text{Bean macro-F1} = (0.9296, 0.9290, 0.9297)$$
$$\text{Bike MAE} = (51.9836, 52.6342, 49.8233)$$
$$\text{Bike RMSE} = (78.2904, 91.1781, 84.3342)$$
$$MAE_{linear} - MAE_{boosting} = 51.9836 - 49.8233 = 2.1603$$
$$RMSE_{boosting} - RMSE_{linear} = 84.3342 - 78.2904 = 6.0438$$

Result. Bike differences use rental-count units. Boosting has lower MAE but higher RMSE than linear. These are arithmetic comparisons of reported metrics, not new significance tests.

Given. Historical Bike weighted-minus-linear operating-loss differences; adjusted conditional intervals; precision target = 5 weighted-count units/hour.

$$\textit{Quality profile: } d = +7.5731,\ \textit{interval} = [3.0698,\ 12.1578]$$
$$d_{max} = \max(7.5731 - 3.0698,\ 12.1578 - 7.5731) = 4.5847 \leq 5$$
$$\textit{Resource profile: } d = -14.0540,\ \textit{interval} = [-19.1464,\ -9.3371]$$
$$d_{max} = \max(5.0924,\ 4.7169) = 5.0924 > 5$$

Result. The quality profile had higher loss than linear but met the precision target. The resource profile had lower loss than linear but missed that precision target. Direction and precision answer different questions.

The corresponding generic precision calculation is:

$$d_{max} = max(\hat{\Delta} - L,\ U - \hat{\Delta}) \quad \text{(T18)}$$

Here L and U are lower and upper interval endpoints in loss units, and $\hat{\Delta}$ is the paired mean loss estimate. d_{max} is an endpoint-distance diagnostic, not a proof of coverage or a probability that the estimate is correct. The historical intervals are approximate and conditional on fitted models and group/calendar-block assumptions. The 16 primary contrasts used 20,000 draws and family error 0.05/16; a separate 40,000-draw calculation checked endpoint stability.

Given. Historical Bean quality-profile estimate d = 0.0011; interval = [−0.0099, 0.0129]; precision target = 0.01.

$$d_{max} = \max[0.0011 - (-0.0099),\ 0.0129 - 0.0011]$$
$$= \max(0.0110,\ 0.0118) = 0.0118 > 0.01$$

Result. The result is inconclusive at the required precision. Identical pipelines yield zero differences by identity, not non-inferiority evidence for different policies.

To reconstruct an interval from raw evidence, define case loss differences $d_i = \ell_i(\text{weighted}) - \ell_i(\text{comparator})$, calculate their row-weighted mean, resample the declared dependence units, and recompute that mean for every draw. Bean uses group-aware resampling; Bike uses month-stratified circular calendar blocks and retains observed gaps. Generic percentile endpoints for a two-sided nominal error α_c are:

$$[L, U] = [Quantile_{\alpha_c/2}(\hat{\Delta}^*),\ Quantile_{1-\alpha_c/2}(\hat{\Delta}^*)] \quad \text{(T19)}$$

Δ^* denotes the collection of resampled paired estimates, not a new model fit. Fix seed, draw count, quantile convention and dependence method. This describes an approximate percentile bootstrap, not a verified finite-sample coverage guarantee. Case-level predictions and private stage inputs are absent from the public manuscript repository; printed aggregate values are insufficient to regenerate these bootstrap draws. Do not invent missing records.

The historical warm CPU proxy multiplies measured seconds per 1,000 requests by USD 0.132/node-hour and divides by 3,600 seconds/hour. Seconds per request were derived from batch timing under the stated workload. This partial conversion excludes setup, idle/support infrastructure, networking and other charges; it is not Equation (2)'s complete cost or a reconciled invoice. Internal cloud refits and consistency checks are not independent academic validation.

11. Check your calculations and record the decision

A reader should finish with a decision record containing: scenario/population and split manifests; immutable candidate/reference identities; metric/oracle and failure conventions; every observed quality and bound; paired

counts or resampling recipe; error allocation and assumptions; unit-normalized cost ledger and bound target; operational definitions and allowances; complete admission reasons; ranking/tie rule; selected policy or explicit no-selection outcome; held-out confirmation; actual-cost reconciliation; source and artifact hashes.

Check sign and scale first. Higher-is-better quality uses reference-minus-candidate degradation. Lower-is-better simulated loss uses candidate-minus-reference when reporting adverse positive differences. Proportions, percentage points and relative percentages differ. Currency/request must become currency/horizon before comparison with the budget. Milliseconds, seconds, MiB and GiB need explicit conversions; 1 GiB=1,024 MiB, whereas a decimal GB is not the same quantity.

Check evidence next. Missing records are not zeros; censored timeouts are not fast successes; duplicated tasks are not independent sample size; sampled peak memory is not a deterministic maximum; warm batch latency is not interactive latency; an expected-cost upper confidence bound is not a future-invoice bound. Store full-precision intermediate values so rounding does not change a gate.

Finally check the inference claim. Nonsignificance is not equivalence or non-inferiority; clipping is not uncertainty reduction; bootstrap repetition is not more independent data; reference identity is not evidence for a different model; minimizing over a finite set is not global optimality. Quality preservation with lower complete cost requires its own frozen, held-out empirical validation. This companion provides calculation instructions, not that validation.

12. Final equation-use checklist

Final use checklist: calculate the task metric with (1), complete horizon cost with (2), justified bounds and admission with (3), or the separately declared weighted route (A1)–(A3). Record inputs, full-precision arithmetic, units, decision reasons and held-out confirmation. If essential evidence is absent, report the limitation rather than filling in a favorable value.

Keep the worked examples labeled as teaching calculations. They explain the method; they do not establish real-model cost savings or the empirical validity of the proposed optimizer.

References and research assistance

[1] Pimpa, P. (2026). Measuring AI Performance and Selecting Models Under Quality and Cost Constraints: Framework Proposal and Empirical Case Studies. Version 1.0.0. Independent Research Manuscript — Not Peer Reviewed. https://github.com/4d696e6b/ai-model-selection-research

[2] scikit-learn developers. f1_score documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

[3] scikit-learn developers. mean_absolute_error documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.mean_absolute_error.html

[4] scikit-learn developers. root_mean_squared_error documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.root_mean_squared_error.html

[5] Clopper, C. J., and Pearson, E. S. (1934). The use of confidence or fiducial limits illustrated in the case of the binomial. Biometrika, 26(4), 404–413. https://doi.org/10.1093/biomet/26.4.404

[6] SciPy developers. scipy.stats.beta documentation, particularly the inverse cumulative-distribution function ppf. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.beta.html

[7] Hoeffding, W. (1963). Probability Inequalities for Sums of Bounded Random Variables. Journal of the American Statistical Association, 58(301), 13–30. https://doi.org/10.1080/01621459.1963.10500830

Technical documentation was checked on 13 October 2026. Current documentation versions are not claimed to be the versions used in the historical Azure study. Publisher metadata for [5] and [7] does not imply that inaccessible full texts were inspected. The original manuscript contains the broader related-work and dataset references.

AI assistance supported the preparation, equation explanations, numerical verification and editing of this companion. Pacharapol Pimpa is the author and remains responsible for verification and interpretation. This independent technical companion is not peer reviewed and does not imply university endorsement. No new AI model experiments, cloud jobs or measured research findings were created for it.

Appendix A. Equation-to-calculation checklist

Equation (1): require aligned inputs, reference outcomes and recorded outputs; apply the frozen task metric; report observed quality, units and sampling design. T1–T7 supply common metric and operating-loss calculations.

Equation (2): require horizon, currency, non-overlapping billable categories, representative expected usage, prices and allocated setup/support; convert every unit and retain failed-policy costs. T8 estimates mean usage; T13–T15 give optional upper-bound constructions under explicit assumptions.

Equation (3): require complete comparable evidence, absolute and reference-relative quality bounds, a declared cost allowance and all resource limits; apply every gate and a deterministic cost ordering. T9–T12 make a binary-quality route explicit; T16 and T17 explain one latency allowance and resource tie-breaking.

Equation (A1): require a criterion measurement and distinct good/bad anchors in matching units; normalize then clip. Equation (A2): require original feasibility and nonnegative weights summing to one; calculate score points. Equation (A3): require a nonempty feasible set, global maximum, ε tolerance and fixed member order; form the near-optimal set and select one member.

T18–T19 explain historical precision diagnostics and approximate paired-bootstrap endpoints. Their outputs cannot be reconstructed from the printed paper alone when private case-level records are absent. The complete worked example follows the proposed route; it is not a report of the historical weighted experiment.

Appendix B. Exact binary-bound calculator recipe

Use this recipe with supplied integer paired counts. It performs arithmetic on records; it does not run an AI model or establish that the sampling assumptions hold. A scientific implementation also needs the frozen evidence contract, operational accounting, exception handling and independent validation.

1. Validate n>0; x_{ref} and $x_{candidate}$ between 0 and n; all $n_{00}, n_{01}, n_{10}, n_{11}$ nonnegative integers; their sum equals n; $x_{ref}=n_{11}+n_{10}$; $x_{candidate}=n_{11}+n_{01}$; 0<e<1.

2. If $x_{candidate}=0$, set the absolute lower bound to zero; otherwise calculate beta.ppf(e,$x_{candidate}$, $n-x_{candidate}+1$).

3. If the two policies are exactly identical with the same records, set degradation to zero by identity. Otherwise compute upper discordance as one if n_{10}=n, else beta.ppf(1−e/2,$n_{10}+1$,n−n_{10}).

4. Compute lower discordance as zero if $n_{01}=0$, else beta.ppf(e/2,n_{01}, $n-n_{01}+1$). Subtract lower discordance from upper discordance to obtain the upper degradation bound.

5. Compare the unrounded absolute lower bound with q_{min} and the unrounded upper degradation bound with δ. Combine these booleans with validated budget and resource booleans using logical AND. Missing evidence is an explicit status, never an automatically true boolean.

6. Sort admitted candidates by the frozen cost/tie key. Return the first candidate if the list is nonempty; otherwise return no supported selection. Preserve the evidence and reasons for all registry entries.

7. On new held-out records, repeat the prespecified confirmation with its own declared allocation; do not reuse the held-out sample to modify the candidate list, thresholds or policy.

Appendix C. Derive the worked example's operational allowances

The operational values in Section 9 can be derived under the following additional teaching assumptions. These are constructed inputs, not measured requests. Assume 1,000 fixed-size independent, identically distributed cost-profiling requests and 1,000 independent future requests from the same stationary workload. All tariffs and setup/support allocations are fixed. Quality evaluations include failures caused by the frozen caps. No retry or ancillary charge lies outside the variable-cost ledger.

For one aggregate variable-cost category, price is one USD per USD of request charge. Mean observed variable charges are A=0.001, B=0.003, C=0.0005 and R=0.005 USD/request. Known, policy-enforced per-request cost caps are respectively 0.003, 0.006, 0.001 and 0.009 USD/request. These caps are assumptions about the complete billing policy, not empirical sample maxima. Allocate η=0.0025 to each candidate's cost-mean bound and β=0.0025 to each future-sum bound, giving at most 0.01 failure allowance for each four-candidate family.

Given. A: m = N = 1,000; $\eta = \beta = 0.0025$; mean charge = USD 0.001/request; enforced charge cap = USD 0.003/request; fixed setup plus support = USD 3.

$$a = \sqrt{[\ln(400)/(2 \times 1{,}000)]} \approx 0.054733283$$
$$\mu_A^U = 0.001 + 0.003a \approx 0.001164199849 \text{ USD/request}$$
$$\text{Future allowance} = 0.003\sqrt{[1{,}000 \ln(400)/2]} \approx 0.164199849 \text{ USD}$$
$$C_{real,N}^U(A) = 3 + 1{,}000\mu_A^U + 0.164199849\ldots \approx 4.328399698 \text{ USD}$$

Result. T13 bounds the mean; T15 adds future workload fluctuation. Retain full precision. The result relies on the stated bounded-charge, independence and stationarity assumptions.

Given. Apply the same method to B, C and R; fixed charges are USD 4, 1.50 and 4.

$$C_{real,N}^U(A, B, C, R) \approx (4.328399698, 7.656799397, 2.109466566, 9.985199095) \text{ USD}$$
$$ceil[C_{real,N}^U] = (5, 8, 3, 10) \text{ USD}$$
$$\text{Projected objective costs} = (4, 7, 2, 9) \text{ USD}$$

Result. Prespecified upward whole-dollar rounding preserves the upper allowance. Nearest-dollar rounding need not. Rank by the declared expected-cost objective, not by these budget allowances.

Given. Per policy: m = 100 independent profiling requests; continuous stationary end-to-end latency; p = 0.95; $\gamma = 0.05/4 = 0.0125$. Include failed attempts under the frozen handling rule.

$$P[\text{Binomial}(100, 0.95) \geq 99] \approx 0.037081209 > \gamma$$
$$P[\text{Binomial}(100, 0.95) \geq 100] = 0.95^{100} \approx 0.005920529 \leq \gamma$$
$$k = 100 \Rightarrow r_{p95}^U = T_{(100)} = \text{sample maximum}$$
$$(r_A^U, r_B^U, r_C^U, r_R^U) = (350, 450, 200, 600) \text{ ms}$$

Result. T16 therefore uses the maximum. These constructed maxima support the exercise's upper p95 allowances only under the stated assumptions.

For memory, assume enforceable allocation caps A=512, B=768, C=256 and R=512 MiB. This is a deterministic limit on the allowed allocation; any cap-triggered crash is already a failed quality outcome. It is not a claim that the sampled process RSS bounds all unseen peaks. A real system must verify how its runtime enforces this cap and which memory definition is being limited.

Given. Failure allowances: quality 0.05; four expected-cost claims together 0.01; future cost 0.01; four latency claims together 0.05.

$$\textit{Total failure allowance} \leq 0.05 + 0.01 + 0.01 + 0.05 = 0.12$$
$$\textit{Simultaneous coverage} \geq 1 - 0.12 = 0.88$$

Result. This union-bound statement is not a combined 95% guarantee. For a combined 95% requirement, allocate a total of 0.05 in advance and recompute all bounds; the admissible set may change. Quality α does not silently cover operational uncertainty.